

Knowledge Representation & Learning

Fitting Binarized Neural Networks

Project presentation – A.Y. 2023-2024

Summary of my work

- ❖ My project
- ❖ Implementation
- ❖ Training phase



- ❖ Testing phase
- ❖ Experiments
- ❖ Demo

My project – Guideline

- ❖ Write a **method** that generates the encoding in **max-sat** of **BNN** layer with **m inputs** and **n outputs**;
- ❖ Use the previous method to generate the encoding in max-sat of a BNN that **stacks a set of binarized layers**;
- ❖ Define a **method for train** a specified BNN **using max-sat**;
- ❖ **Generate train and test** data for **at least 3 binary functions** with **5 inputs** and **5 outputs** and evaluate the performance with different net configurations.



My project – Two concepts together

It is possible to see this project as the **union** between **two concepts**:

1. The **simplest** form of **NN**;
2. The resolution of a **MaxSat problem**.

Indeed



- ❖ The **BNN** is a basic form of NN where both neurons and weights have **values 1 or -1** (True or False);
- ❖ The goal of the project is to find the **model that best fits the data**. This means:
 - To find the best weights configuration, **in order to maximize** the correct predictions of the net.

MaxSat problem

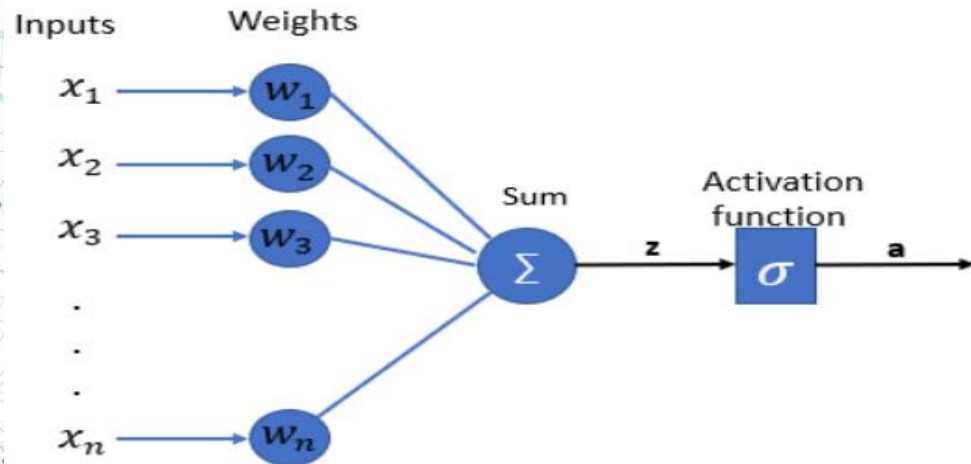
My project – Development phases

During the realization **of my project**, I followed the following **development phases**:

- ❖ **Binarized Neural Networks realization:** **different** architectures and net configurations;
- ❖ **Dataset formation:** **random values** of **1** and **-1**;
- ❖ **MaxSat problem formulation:** **3** algorithms implemented;
- ❖ **Training of the nets:** with **different** net configurations and using **different** MaxSat algorithms;
- ❖ **Testing of the nets:** **standard accuracy** or **balanced accuracy** (based on the dataset).



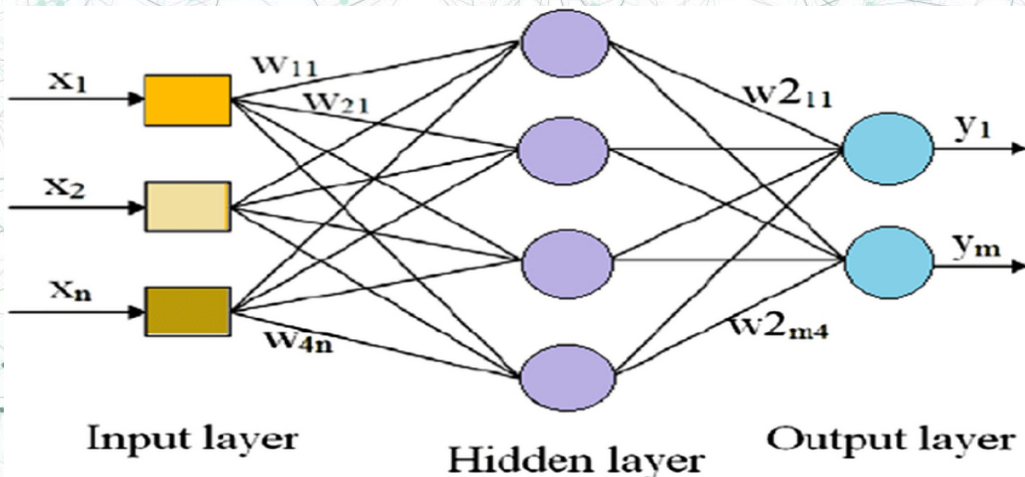
Implementation – Binarized Neural Networks (1)



For the **BNNs** I decided to develop **two** different **versions**:

No hidden layers version

- ❖ Only input and output layers;
- ❖ Single **perceptron** inspiration;
- ❖ Boolean operators: **OR, AND, NOT**.



Hidden layers version

- ❖ At least one hidden layer;
- ❖ **Multilayer perceptron** inspiration;
- ❖ Boolean operators: **XOR, XNOR, NOR, NAND**.

Implementation – Binarized Neural Networks (2)

I implemented **different networks architectures**

The differences concern:

- ❖ number of **input neurons** (m);
- ❖ number of **output neurons** (n);
- ❖ number and type of **binary functions** implemented.

My configurations



- ❖ **m input** neurons, **1 binary function** and **n=1 output** neuron;
- ❖ **m input** neurons, **3 binary functions** and **n=3 output** neurons;
- ❖ **m input** neurons, **5 binary functions** and **n=5 output** neurons.

Fragment of code

```
1 # boolean operator for single perceptron
2
3 def OR(x): # binary function
4     for i in x:
5         if i == 1:
6             return 1
7     return -1
8
9 def AND(x): # binary function
10    for i in x:
11        if i == -1:
12            return -1
13    return 1
14
15 def NOT(x): # unary function
16     if x == [1]:
17         return -1
18     return 1
```

```
1 # boolean operator for multilayer perceptron
2
3 def XOR(x): # exclusive OR [-1,-1] - [1,1] = false
4     for p in range(0, len(x)-1):
5         if x[p] == x[p+1]:
6             return -1
7     return 1
8
9 def XNOR(x): # NOT(XOR): [-1,-1] - [1,1] = true
10    return NOT([XOR(x)])
11
12 def NOR(x): # NOT(OR)
13    return NOT([OR(x)])
14
15 def NAND(x): # NOT(AND)
16    return NOT([AND(x)])
```

**For both single and multilayer
perceptron inspirations**

Implementation – Dataset formation

Fragment of code

```

1 examples = 200;
2 m = 2; # INPUT NEURONS
3 n = 1; # OUTPUT NEURONS
4
5 X = []; # pairs [(x1, ..., xm); y]
6
7 for i in range(examples): # generation of some integers (1 or -1)
8     row = [];
9     for _ in range(0, m):
10         row.append(randrange(-1, 2, 2)) # start, stop, step
11
12     # different y_target based on the logic operation used
13     # EXAMPLE OF SINGLE PERCEPTRON
14     y_target = AND(row)
15     #####
16     #y_target = AND(row)
17     #y_target = NOT(row)
18
19     X.append(tuple([row, y_target]))
20
21 print ("PARAMETERS OF MY NET :-)")
22 print ("-----+")
23 print ("DIM input:", m)
24 print ("DIM output:", n)
25 print("DIM vector weights:", len(X[0][0]), "*", n, "\n")
26 print("Binary function implemented: AND")
27 print ("-----+")
28 print("OKAY, DATASET realized :-) <<")

```

Based on the network configuration, I obtained **different datasets**

The differences concern:

- ❖ **number of rows** present in the dataset (examples);
- ❖ **length of each row** of the dataset (m);
- ❖ **length of each label** of the dataset (n).



Common rules

- ❖ Randomly composed of the **values 1 and -1**;
- ❖ **Training set: 85%, Test set: 15%.**

Implementation – MaxSat problem formulation

- ❖ I decided to develop **3 different methods from scratch**;
- ❖ All these algorithms find the **best weights configuration**;
- ❖ The **underlying logic is different**.

Fragment of code

```

1 # COMMON FUNCTION USED BOTH FROM PERMUTATION MAXSAT AND CLAUSES MAXSAT
2 def bestweight_MaxSat(SAT_weights): # return the weights configuration that more often sat the examples
3
4     weight_max = [] # contains the more frequent weights configuration encountered during training phase - MAXSAT(weights)
5     max = 0 # counter for MAXSAT(weights)
6
7     for wi in SAT_weights:
8         count = 0 # number of occurrences of wi
9         for wj in SAT_weights:
10
11             if wi == wj:
12                 count += 1
13
14             if count > max:
15                 weight_max = wi
16                 max = count
17
18     return weight_max
    
```

1. **Permutation MaxSat**
2. **Naïve Search**
3. **Clauses MaxSat**



Implementation – Permutation MaxSat

Fragment of code

```

4 def Permutation_MaxSat(j): # I repeat this for each j leables
5
6 SAT_weights = []; # list of the weight that fit the data
7
8 for i in range(0, len(X_train)): # I scroll through each example of the training set (from 0 to [len(X_train)-1])
9
10     pos = m; # position used to insert incrementally the value -1 (to m-1 to 0)
11     SAT = False # boolean flag
12     next = False # in case the sat configuration is not found becomes true => I pass to the next example
13
14     weights = list(1 for i in range(0, m)) # list of weight for each examples (from 0 to m-1) => initially all weights are considered sat
15
16     y_pred = pred(weights, X_train[i][0]) # model prediction
17
18     if y_pred == X_train[i][j]: # the model prediction is correct
19         SAT = True
20         SAT_weights.append(weights) # insert the element at the end of the list
21
22     else:
23         while SAT == False & next == False: # the model prediction is not correct
24             pos -= 1 # last position of the list/examine [ 1 1 1 1 -1], [1 1 1 -1 -1], ...
25             weights[pos] = -1 # flip the bit
26
27             permutation_w = built_permutation(weights) # all the possible permutations of weights
28             new_w, SAT = recheck_SAT(weights, i, j, permutation_w) # weights satisfiability recheck for the axample i
29
30             if SAT == False:
31                 if pos > 0:
32                     weights = new_w
33
34                 else: # pos = 0 => stop
35                     next = True
36                     break
37
38             if SAT == True:
39                 SAT_weights.append(new_w)
40
41 return bestweight_MaxSat(SAT_weights) # maxsat weight configuration => is the one that most often SAT the examples

```

Algorithm that search the best weights configuration (i.e with the smallest number of -1), simply using a **list of permutations**

In total I realized **4 functions**:

- ❖ ***Permutation_MaxSat***: general function that returns the **best weights configuration**;
- ❖ ***built_permutation***: auxiliary function that calculates all possible **permutations** of the weights provided as input;
- ❖ ***recheck_SAT***: auxiliary function that performs the **satisfiability recheck** for the new list of weights obtained;
- ❖ ***bestweight_MaxSat***: auxiliary function that returns the weights configuration that **more often** satisfies the examples.

Implementation – Naïve Search

Algorithm that search the best weights configuration (i.e with the smallest number of -1), simply using a **naïve approach**

I realized only **1 function**:

❖ **Naive_Search**, where

- Weights update rule: **difference between predicted output and real output**;
- **Weights normalization**: $w_i \in \{1, -1\}$;
- Parameters setting: both **learning rate** and **epochs** are equal to 1.

Fragment of code

```

4 def Naive_Search(j, epochs): # epochs and lr = 1
5
6     weights = list(1 for i in range(0, m)) # list of weight for each examples (from 0 to m-1) => initially all weights are considered sat
7
8     for _ in range(0, epochs):
9         for i in range(0, len(X_train)): # I scroll all the examples in the training set
10             y_pred = pred(weights, X_train[i][0]) # model prediction
11
12             if y_pred != X_train[i][j]: # prediction wrong
13                 for k in range (0, m): # learning rate = 1
14                     weights[k] += 1* (X_train[i][j] - y_pred) * X_train[i][0][k] # weight updating for every k-element of the data
15
16             for w in range(0, m):
17                 weights[w] = fun(weights[w]) # weights normalization (-1 or 1)
18
19     return weights

```



Implementation – Clauses MaxSat (1)

HARD CONSTRAINTS: $\bigwedge_{i=1}^m (w_i \vee x_i)$

❖ **NOT**

➤ $\infty : \neg w_1 \wedge (x_1 \vee \neg x_1)$

❖ **AND, OR, XOR, XNOR, NOR, NAND**

➤ $\infty : w_1 \vee w_2 \vee \dots \vee w_n \vee \neg w_n$

➤ $\infty : x_1 \vee x_2 \vee \dots \vee x_n \vee \neg x_n$

➤ $\infty : w_1 \vee \dots \vee w_{n-1} \vee x_n \vee \neg x_n$

➤ $\infty : x_1 \vee \dots \vee x_{n-1} \vee w_n \vee \neg w_n$

SOFT CONSTRAINTS: $\{1 : w_i\}_{i=1}^m$

All in CNF

Algorithm that search the best weights configuration (i.e with the smallest number of -1), thanks the use of **PySAT**

In total I realized **3 functions**:

❖ **Clauses_MaxSat**: general function that returns the **best weights configuration**;

❖ **MAXSAT_solver**: auxiliary function which thanks to the implementations of the **hard** and **soft clauses**, returns **all the MaxSat models that satisfy the hard clauses** (from the less to the more expensive);

❖ **bestweight_MaxSat**: auxiliary function that returns the weights configuration that **more often** satisfies the examples.

Implementation – Clauses MaxSat (2)

Fragment of Clauses_ MaxSat function

```

11 for i in range(0, len(X_train)): # I scroll through each example of the training set (from 0 to [len(X_train)-1])
12     SAT = False # boolean flag
13     next = False # in case the sat configuration is not found becomes true => I pass to the next example
14
15     weights = list(1 for i in range(0, m)) # list of weight for each examples (from 0 to m-1) => initially all weights are considered sat
16
17     while SAT == False & next == False:
18         for model in MaxSat_models:
19             match = False # found match between data and maxsat models
20
21             for w in range(0, m): # check weights [from w0 to m-1] => if (m=2): w0, w1
22                 weights[w] = fun(model[w]) # insertion of the weights maxsat model evaluated under f in the weights list
23
24             for x in range(m, 2*m): # control all training variables x [from m to (m*2)-1] => if (m=2): x2, x3
25
26                 # [i]: example i, [0]: data, [m-x]: element m-x of the data
27                 if X_train[i][0][m-x] == fun(model[x]): # check all the element in the data[0] => m=2 (2 to 3): elem 2-2 = 0, elem 2-3 = 1
28                     match = True
29                 else:
30                     break
31
32             if match == True:
33                 y_pred = pred(weights, X_train[i][0])
34
35                 if y_pred == X_train[i][j]:
36                     SAT = True
37                     SAT_weights.append(weights)
38                     break
39
40             if SAT == False: # stop
41                 next = True
42                 break

```

```

16 #####
17 # HARD CONSTRAINTS
18 #####
19 def MAXSAT_solver(m):
20     all_maxsat_sol = []
21     rc2 = RC2(WCNF()) # passing an empty WCNF() formula
22
23     # 1    2
24     # w1   x1
25
26     # NOT function
27     if m == 1: # number of input neurons
28         rc2.add_clause([-1]) # FIRST CLAUSE: w1 = -1
29         rc2.add_clause([2, -2]) # SECOND CLAUSE: x1 or !x1
30
31     # -----
32     # for AND, OR and composition boolean function
33
34     # 1    2    3    4
35     # w1   w2   x1   x2
36     # assumptions: xn = 4, !xn = -4
37     # | | | | | # wn = 2, !wn = -2
38
39     if m == 2:
40         rc2.add_clause([1, 2, -2]) # w1 or w2 or !w2
41         rc2.add_clause([3, 4, -4]) # x1 or x2 or !x2
42         rc2.add_clause([1, 4, -4]) # w1 or x2 or !x2
43         rc2.add_clause([3, 2, -2]) # x1 or w2 or !w2

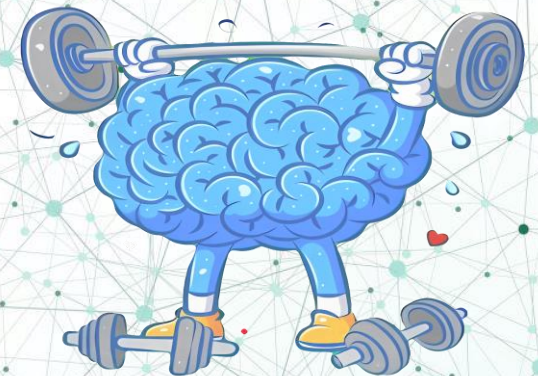
```

```

107 #####
108 # SOFT CONSTRAINTS
109 #####
110 for i in range (0, m): # from 0 to m-1
111     rc2.add_clause([i+1], weight=1)
112
113
114 for m in rc2.enumerate(): # to enumerate all the maxsat solution (i.e all the models) computed
115     all_maxsat_sol.append(m)
116     # print('model {0} has cost {1}'.format(m, rc2.cost)) # print all the models from the less costly to the most
117
118 return all_maxsat_sol # contains all the possible MaxSat solution that sat the hard clauses

```

Hard & soft constraints



Training phase

Each net configuration is trained using each of the 3 developed algorithms: **Permutation Maxsat, Naïve Search and Clauses**

MaxSat

The result at the end of each training is composed by:

- ❖ **Best weights vector** obtained from the algorithm;
- ❖ **% of SAT elements** of the weights configuration;
- ❖ **% of UNSAT elements** of the weights configuration.

**Output obtained after the training
phase for the Clauses MaxSat
algorithm**

```
TRAINING OF THE NET =====>
A little patience the network is learning ... :-)
+-----+
Best weights obtained with the Clauses MaxSat: [[1, 1, 1, 1, 1], [1, 1, 1, 1, 1], [1, 1, 1, 1, 1], [1, 1, 1, 1, 1], [1, 1, 1, 1, 1]]
% of SAT elements of the weights configuration: 100.00 %
% of UNSAT elements of the weights configuration: 00.00 %
```

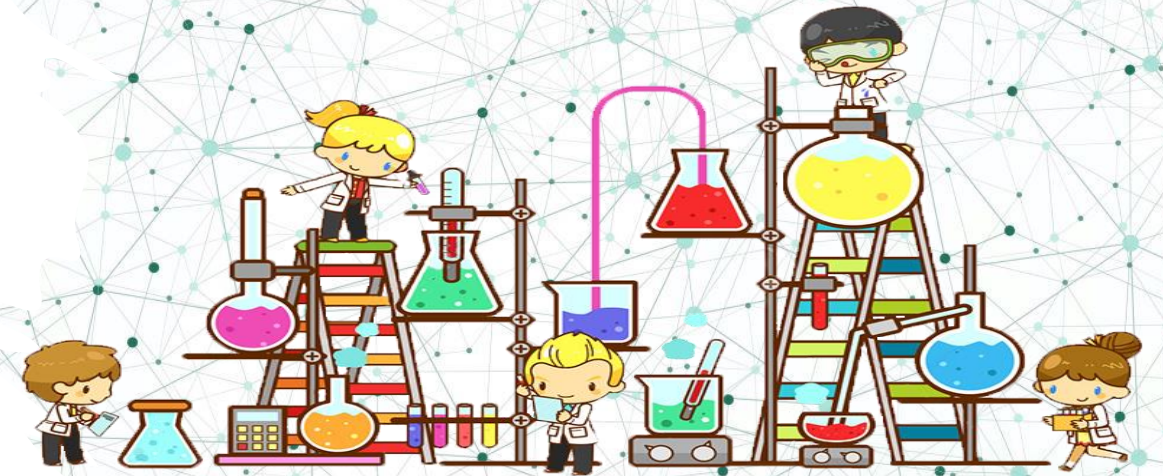

Testing phase – Environment settings

During the testing phase, I tested the behavior of **my 3 MaxSat algorithms**:

- ❖ On **different datasets**;
 - ❖ With **different net configurations**.
- }
- To try to understand in depth the behavior of each method

In each experiment the changes concern:

- ❖ number of **examples** in the dataset;
- ❖ number of **input neurons** of the net;
- ❖ number of **output neurons** of the net;
- ❖ number and type of **binary functions** implemented.



- ➔
- ❖ **Method**: comparison of the results obtained;
 - ❖ **Metrics**: Accuracy (standard or balanced).



Experiments – Results tables (1)

NO HIDDEN LAYERS VERSION – m input neurons, 1 binary function and n=1 output neuron

NOT			
Single perceptron: NOT, ex = 50, m=1, n=1	[-1] , sat= 0%, unsat = 100%, ac = 100%	epochs = 1: [-1] , sat= 100%, unsat = 0%, ac = 100%	[-1] , sat= 0%, unsat = 100%, ac = 100%
Single perceptron: NOT, ex = 200, m=1, n=1	[-1] , sat= 0%, unsat = 100%, ac = 100%	epochs = 1: [-1] , sat= 100%, unsat = 0%, ac = 100%	[-1] , sat= 0%, unsat = 100%, ac = 100%
Single perceptron: NOT, ex = 200, m=1, n=1	[-1] , sat= 0%, unsat = 100%, ac = 100%	epochs = 1: [-1] , sat= 100%, unsat = 0%, ac = 100%	[-1] , sat= 0%, unsat = 100%, ac = 100%
Single perceptron: NOT, ex = 1000, m=1, n=1	[-1] , sat= 0%, unsat = 100%, ac = 100%	epochs = 1: [-1] , sat= 100%, unsat = 0%, ac = 100%	[-1] , sat= 0%, unsat = 100%, ac = 100%
AND			
Single perceptron: AND, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 100%	epochs = 1: [1, 1] , sat= 100%, unsat = 0%, acb = 100%	[1, 1] , sat= 100%, unsat = 0%, acb = 100%
Single perceptron: AND, ex = 200, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 86,54%	epochs = 1: [1, 1, -1] , sat= 66,67%, unsat = 33,33%, acb = 78,85%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 86,54%
Single perceptron: AND, ex = 200, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 74,14%	epochs = 1: [1, 1, -1, -1, -1] , sat= 60%, unsat = 40%, acb = 22,41%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 74,14%
Single perceptron: AND, ex = 1000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 76,03%	epochs = 1: [1, -1, 1, 1, 1] , sat= 80%, unsat = 20%, acb = 75,68%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 76,03%
Single perceptron: AND, ex = 5000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,99%	epochs = 1: [-1, -1, 1, 1, -1] , sat= 40%, unsat = 60%, acb = 25,28%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,99%
Single perceptron: AND, ex = 10000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 76,44%	epochs = 1: [-1, -1, -1, -1, -1] , sat= 0%, unsat = 100%, acb = 23,56%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 76,44%
Single perceptron: AND, ex = 10000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,65%	epochs = 1: [1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,65%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,65%
Single perceptron: AND, ex = 50000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,99%	epochs = 1: [1, 1, 1, -1, -1] , sat= 60%, unsat = 40%, acb = 75,56%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,99%
OR			
Single perceptron: OR, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 65,62%	epochs = 1: [1, 1] , sat= 100%, unsat = 0%, acb = 65,62%	[1, 1] , sat= 100%, unsat = 0%, acb = 65,62%
Single perceptron: OR, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 66,67%	epochs = 1: [1, -1] , sat= 50%, unsat = 50%, acb = 64,58%	[1, 1] , sat= 100%, unsat = 0%, acb = 66,67%
Single perceptron: OR, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 68,18%	epochs = 1: [1, -1] , sat= 50%, unsat = 50%, acb = 75%	[1, 1] , sat= 100%, unsat = 0%, acb = 68,18%
Single perceptron: OR, ex = 10 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,41 %	epochs = 1: [1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,41%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,41%
Single perceptron: OR, ex = 10 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,13%	epochs = 1: [-1, 1, -1] , sat= 33,33%, unsat = 66,67%, acb = 21,48%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,13%
Single perceptron: OR, ex = 10 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,66%	epochs = 1: [1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,66%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,66%
Single perceptron: OR, ex = 10 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,55%	epochs = 1: [1, 1, -1, -1, 1] , sat= 60%, unsat = 40%, acb = 76,58%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,55%
Single perceptron: OR, ex = 50 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,93 %	epochs = 1: [1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,93 %	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,93 %

❖ 1st column: net architecture

❖ 2nd column: Permutation MaxSat

❖ 3rd column: Naïve Search

❖ 4th column: Clauses MaxSat



Experiments – Results tables (1)

NO HIDDEN LAYERS VERSION – m input neurons, 1 binary function and n=1 output neuron

standard

NOT			
Single perceptron: NOT, ex = 50, m=1, n=1	[-1] , sat= 0%, unsat = 100%, ac = 100%	epochs = 1: [-1] , sat= 100%, unsat = 0%, ac = 100%	[-1] , sat= 0%, unsat = 100%, ac = 100%
Single perceptron: NOT, ex = 200, m=1, n=1	[-1] , sat= 0%, unsat = 100%, ac = 100%	epochs = 1: [-1] , sat= 100%, unsat = 0%, ac = 100%	[-1] , sat= 0%, unsat = 100%, ac = 100%
Single perceptron: NOT, ex = 200, m=1, n=1	[-1] , sat= 0%, unsat = 100%, ac = 100%	epochs = 1: [-1] , sat= 100%, unsat = 0%, ac = 100%	[-1] , sat= 0%, unsat = 100%, ac = 100%
Single perceptron: NOT, ex = 1000, m=1, n=1	[-1] , sat= 0%, unsat = 100%, ac = 100%	epochs = 1: [-1] , sat= 100%, unsat = 0%, ac = 100%	[-1] , sat= 0%, unsat = 100%, ac = 100%
AND			
Single perceptron: AND, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 100%	epochs = 1: [1, 1] , sat= 100%, unsat = 0%, acb = 100%	[1, 1] , sat= 100%, unsat = 0%, acb = 100%
Single perceptron: AND, ex = 200, m=2, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 86,54%	epochs = 1: [1, 1, -1] , sat= 66,67%, unsat = 33,33%, acb = 78,85%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 86,54%
Single perceptron: AND, ex = 200, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 74,14%	epochs = 1: [1, 1, -1, -1, -1] , sat= 60%, unsat = 40%, acb = 22,41%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 74,14%
Single perceptron: AND, ex = 1000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 76,03%	epochs = 1: [1, -1, 1, 1, 1] , sat= 80%, unsat = 20%, acb = 75,68%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 76,03%
Single perceptron: AND, ex = 5000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,99%	epochs = 1: [-1, -1, 1, 1, -1] , sat= 40%, unsat = 60%, acb = 25,28%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,99%
Single perceptron: AND, ex = 10000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 76,44%	epochs = 1: [-1, -1, -1, -1, -1] , sat= 0%, unsat = 100%, acb = 23,56%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 76,44%
Single perceptron: AND, ex = 10000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,65%	epochs = 1: [1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,65%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,65%
Single perceptron: AND, ex = 50000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,99%	epochs = 1: [1, 1, 1, -1, -1] , sat= 60%, unsat = 40%, acb = 75,56%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,99%
OR			
Single perceptron: OR, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 65,62%	epochs = 1: [1, 1] , sat= 100%, unsat = 0%, acb = 65,62%	[1, 1] , sat= 100%, unsat = 0%, acb = 65,62%
Single perceptron: OR, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 66,67%	epochs = 1: [1, -1] , sat= 50%, unsat = 50%, acb = 64,58%	[1, 1] , sat= 100%, unsat = 0%, acb = 66,67%
Single perceptron: OR, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 68,18%	epochs = 1: [1, -1] , sat= 50%, unsat = 50%, acb = 75%	[1, 1] , sat= 100%, unsat = 0%, acb = 68,18%
Single perceptron: OR, ex = 10 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,41 %	epochs = 1: [1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,41%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,41%
Single perceptron: OR, ex = 10 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,13%	epochs = 1: [-1, 1, -1] , sat= 33,33%, unsat = 66,67%, acb = 21,48%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 79,13%
Single perceptron: OR, ex = 10 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,66%	epochs = 1: [1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,66%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,66%
Single perceptron: OR, ex = 10 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,55%	epochs = 1: [1, 1, -1, -1, 1] , sat= 60%, unsat = 40%, acb = 76,58%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,55%
Single perceptron: OR, ex = 50 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,93 %	epochs = 1: [1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,93 %	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 75,93 %

balanced



To test the **robustness** of the algorithms (the influence of the runtime)
some experiments were **repeated more times**.

Included in the notebook



Experiments – Results tables (2)

HIDDEN LAYERS VERSION – m input neurons, 1 binary function and n=1 output neuron

NAND			
Multi layers: NAND, ex = 200, m=2, n=1	[1, -1] , sat= 50%, unsat = 50%, acb = 70,45 %	epochs = 1: [1, -1] , sat= 50%, unsat = 50%, acb = 70,45 %	[1, -1] , sat= 50%, unsat = 50%, acb = 70,45 %
Multi layers: NAND, ex = 800, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 23,02%	epochs = 1: [1, 1, 1] , sat= 66,67%, unsat = 33,33%, acb = 15,87%	[1, 1, 1] , sat= 100%, unsat = 100%, acb = 23,02%
Multi layers: NAND, ex = 1200, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 18,93%	epochs = 1: [1, 1, 1] , sat= 100%, unsat = 0%, acb = 18,93%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 18,93%
Multi layers: NAND, ex = 8000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 23,42%	epochs = 1: [-1, -1, -1] , sat= 0%, unsat = 100%, acb = 76,58%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 23,42%
Multi layers: NAND, ex = 10 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 21,61%	epochs = 1: [1, 1, -1] , sat= 33,33%, unsat = 66,67%, acb = 80,61%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 21,61%
Multi layers: NAND, ex = 10 000, m=5, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 19,98%	epochs = 1: [1, -1, 1] , sat= 66,67%, unsat = 33,33%, acb = 21,93%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 19,98%
Multi layers: NAND, ex = 10 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 24,59 %	epochs = 1: [-1, 1, 1, 1, 1] , sat= 80%, unsat = 20%, acb = 24,45%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 24,59%
Multi layers: NAND, ex = 50 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 24,07%	epochs = 1: [1, 1, -1, -1, -1] , sat= 40%, unsat = 60%, acb = 75,78%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 24,07%
NOR			
Multi layers: NOR, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 36%	epochs = 1: [-1, -1] , sat= 0%, unsat = 100%, acb = 100%	[1, 1] , sat= 100%, unsat = 0%, acb = 36%
Multi layers: NOR, ex = 500, m=3, n=1	[1, 1] , sat= 100%, unsat = 0%, acb = 26,92%	epochs = 1: [-1, -1] , sat= 0%, unsat = 100%, acb = 100%	[1, 1] , sat= 100%, unsat = 0%, acb = 26,92%
Multi layers: NOR, ex = 800, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 23,48%	epochs = 1: [-1, -1, 1] , sat= 33,33%, unsat = 66,67%, acb = 76,52%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 23,48%
Multi layers: NOR, ex = 1200, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 21,43%	epochs = 1: [-1, -1, -1] , sat= 0%, unsat = 100%, acb = 78,57%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 21,43%
Multi layers: NOR, ex = 1200, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 23,03%	epochs = 1: [1, -1, 1] , sat= 66,67%, unsat = 33,33%, acb = 25,66%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 23,03 %
Multi layers: NOR, ex = 8000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 25,96%	epochs = 1: [1, 1, -1] , sat= 66,67%, unsat = 33,33%, acb = 24,36%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 25,96%
Multi layers: NOR, ex = 10 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 21,82%	epochs = 1: [-1, -1, -1] , sat= 0%, unsat = 100%, acb = 78,18%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 21,82%
Multi layers: NOR, ex = 10 000, m=5, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 22,59%	epochs = 1: [1, 1, -1] , sat= 66,67%, unsat = 33,33%, acb = 22,06%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 22,59%
Multi layers: NOR, ex = 10 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 25,79%	epochs = 1: [-1, -1, 1, -1, -1] , sat= 20%, unsat = 80%, acb = 74,17%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 25,79%
Multi layers: NOR, ex = 50 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 24,16%	epochs = 1: [1, 1, 1, 1, -1] , sat= 80%, unsat = 20%, acb = 24,20%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 24,16%
XOR			
Multi layers: XOR, ex = 200, m=2, n=1	[1, -1] , sat= 50%, unsat = 50%, ac = 86,67%	epochs = 1: [-1, 1] , sat= 50%, unsat = 50%, ac = 63,33%	[1, 1] , sat= 50%, unsat = 50%, ac = 86,67 %
Multi layers: XOR, ex = 800, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 48,43 %	epochs = 1: [1, 1, 1] , sat= 100%, unsat = 0%, acb = 48,43 %	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 48,43 %
Multi layers: XOR, ex = 10 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 47,24 %	epochs = 1: [1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 47,24 %	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 47,24 %
Multi layers: XOR, ex = 50 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 51,15%	epochs = 1: [1, -1, 1] , sat= 66,67%, unsat = 33,33%, acb = 51,32%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 51,15%
Multi layers: XOR, ex = 80 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 50,46 %	epochs = 1: [-1, 1, -1] , sat= 33,33%, unsat = 66,67%, acb = 49,77 %	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 50,46%
Multi layers: XOR, ex = 80 000, m=3, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 50,14%	epochs = 1: [-1, -1, -1] , sat= 0%, unsat = 100%, acb = 49,89 %	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 50,14%
XNOR			
Multi layers: XNOR, ex = 200, m=2, n=1	[1, 1] , sat= 100%, unsat = 0%, ac = 70,00%	epochs = 1: [-1, -1] , sat= 0%, unsat = 100%, ac = 83,33%	[1, 1] , sat= 100%, unsat = 0%, ac = 70,00%
Multi layers: XNOR, ex = 10 000, m=3, n=1	[1, 1] , sat= 100%, unsat = 0%, ac = 70,00%	epochs = 1: [-1, -1] , sat= 0%, unsat = 100%, ac = 90,00%	[1, 1] , sat= 100%, unsat = 0%, ac = 70,00%
Multi layers: XNOR, ex = 10 000, m=5, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 49,70%	epochs = 1: [-1, -1, 1] , sat= 33,33%, unsat = 66,67%, acb = 52,33%	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 49,70%
Multi layers: XNOR, ex = 50 000, m=3, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 52,00%	epochs = 1: [1, 1, 1, 1, -1] , sat= 80%, unsat = 20%, acb = 48,96%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 52,00%
Multi layers: XNOR, ex = 50 000, m=5, n=1	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 49,92%	epochs = 1: [-1, 1, -1] , sat= 33,33%, unsat = 66,67 %, acb = 49,89 %	[1, 1, 1] , sat= 100%, unsat = 0%, acb = 49,92%
Multi layers: XNOR, ex = 50 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 48,97%	epochs = 1: [-1, 1, 1, 1, 1] , sat= 80%, unsat = 20%, acb = 50,83%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 48,97%
Multi layers: XNOR, ex = 80 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 49,48%	epochs = 1: [-1, 1, -1, -1, -1] , sat= 20%, unsat = 80%, acb = 50,63%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 49,48%
Multi layers: XNOR, ex = 200 000, m=5, n=1	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 50,95%	epochs = 1: [1, 1, -1, -1, -1] , sat= 40%, unsat = 60%, acb = 48,9%	[1, 1, 1, 1, 1] , sat= 100%, unsat = 0%, acb = 50,95%

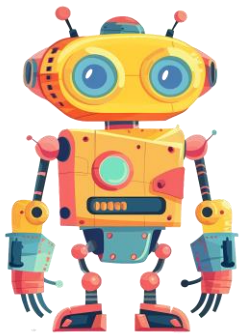


Experiments – Results tables (3)

NO HIDDEN LAYERS VERSION – m input neurons, 3–5 binary functions and $n=3-5$ output neurons

Single perceptron ($n=3$)			
AND-AND-AND, ex. 200, $m=3$, $n=3$	sat=100%, usat=0%, acb=72%	epochs=1: sat=66,67%, usat=33,33%, acb=78%	sat=100%, usat=0%, acb=72%
OR-OR-OR, ex. 200, $m=3$, $n=3$	sat=100%, usat=0%, acb=83,93%	epochs=1: sat=66,67%, usat=33,33%, acb=75%	sat=100%, usat=0%, acb=83,93%
OR-AND-OR, ex. 200, $m=3$, $n=3$	sat=100%, usat=0%, acb=61,57%	epochs=1: sat=88,89%, usat=11,11%, acb=67,13%	sat=100%, usat=0%, acb=61,57%
AND-OR-AND, ex. 200, $m=3$, $n=3$	sat=100%, usat=0%, acb=67,65%	epochs=1: sat=77,78%, unsat=22,22%, acb=52%	sat=100%, usat=0%, acb=67,65%
OR-OR-OR, ex. 200, $m=5$, $n=3$	sat=100%, usat=0%, acb=81,03%	epochs=1: sat=60%, unsat=40, acb=81,03%	sat=100%, usat=0%, acb=81,03%
OR-OR-OR, ex. 200, $m=5$, $n=3$	sat=100%, usat=0%, acb=79,31%	epochs=1: sat=80%, unsat=20, acb=68,97%	sat=100%, usat=0%, acb=79,31%
Single perceptron ($n=5$)			
AND-AND-AND-AND-AND, ex. 200, $m=5$, $n=5$	sat=100%, usat=0%, acb=80,36%	epochs=1: sat=60%, usat=40%, acb=75%	sat=100%, usat=0%, acb=80,6%
OR-OR-OR-OR-OR, ex. 200, $m=5$, $n=5$	sat=100%, usat=0%, acb=82,14%	epochs=1: sat=60%, usat=40%, acb=80,36%	sat=100%, usat=0%, acb=82,14%
OR-AND-OR-AND-OR, ex. 200, $m=5$, $n=5$	sat=100%, usat=0%, acb= 55,09%	epochs=1: sat=60%, usat=40%, acb=46,11%	sat=100%, usat=0%, acb= 55,09%
AND-OR-AND-OR-AND, ex. 200, $m=5$, $n=5$	sat=100%, usat=0%, acb= 61,92%	epochs=1: sat=48%, usat=52%, acb= 47,96%	sat=100%, usat=0%, acb= 61,92 %

❖ **Different** number and **combinations** of binary functions.



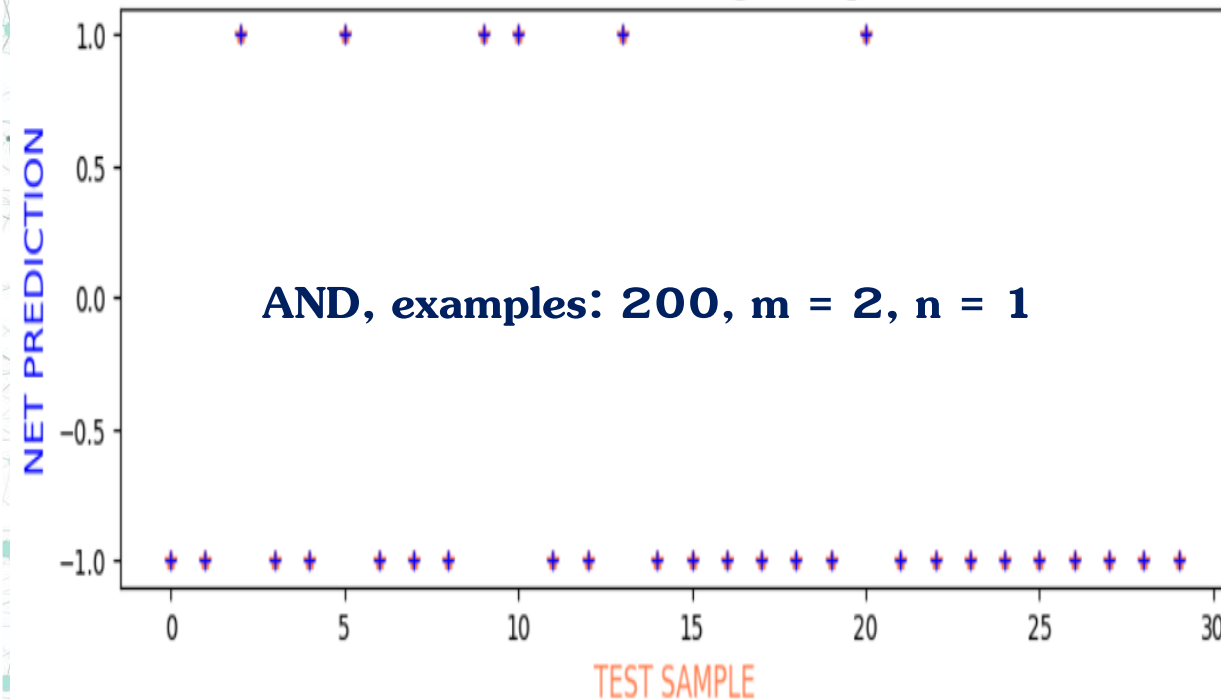
Experiments – Results tables (4)

HIDDEN LAYERS VERSION – m input neurons, 3-5 binary functions and n=3-5 output neurons

Multi layer (n=3)			
NAND-NAND-NAND, ex. 200, m=3, n=3	sat=100%, usat=0%, acb= 22,22%	epochs=1: sat=0%, usat=100%, acb= 77,78%	sat=100%, usat=0%, acb= 22,22%
NOR-NOR-NOR, ex. 200, m=3, n=3	sat=100%, usat=0%, acb= 21,43%	epochs=1: sat=33,33%, usat=66,67%, acb= 75,00%	sat=100%, usat=0%, acb= 21,45%
XOR-NAND-XOR, ex. 200, m=3, n=3	sat=100%, usat=0%, acb= 43,21%	epochs=1: sat=11,11%, usat=88,89%, acb= 58,84%	sat=100%, usat=0%, acb= 43,21%
NAND-XOR-NAND, ex. 200, m=3, n=3	sat=100%, usat=0%, acb= 44,95%	epochs=1: sat=22,22%, usat=77,78%, acb= 54,69%	sat=100%, usat=0%, acb= 44,95%
XNOR-NOR-XNOR, ex. 200, m=3, n=3	sat=100%, usat=0%, acb= 59,92%	epochs=1: sat=33,33%, usat=66,67%, acb= 73,96%	sat=100%, usat=0%, acb= 59,92%
NAND-NOR-NAND, ex. 200, m=3, n=3	sat=100%, usat=0%, acb= 41,67%	epochs=1: sat=33,33%, usat=66,67%, acb= 39,35%	sat=100%, usat=0%, acb= 41,67 %
NAND-XNOR-NOR, ex. 200, m=3, n=3	sat=100%, usat=0%, acb= 43,78%	epochs=1: sat=44,44%, usat=55,56%, acb= 66,55%	sat=100%, usat=0%, acb= 43,78 %
1. NAND-NAND-NAND			
NAND-NAND-NAND, ex. 1000, m=3, n=3	sat=100%, usat=0%, acb= 24,81%	epochs=1: sat=0%, usat=100%, acb= 75,19%	sat=100%, usat=0%, acb= 24,81%
NAND-NAND-NAND, ex. 5000, m=5, n=3	sat=100%, usat=0%, acb= 21,21%	epochs=1: sat=0%, usat=100%, acb= 78,79%	sat=100%, usat=0%, acb= 21,21%
NAND-NAND-NAND, ex. 10 000, m=5, n=3	sat=100%, usat=0%, acb= 25,48%	epochs=1: sat=40%, usat=60%, acb= 75,00%	sat=100%, usat=0%, acb= 25,48%
NAND-NAND-NAND, ex. 10 000, m=5, n=3	sat=100%, usat=0%, acb= 23,85%	epochs=1: sat=40%, usat=60%, acb= 75,43%	sat=100%, usat=0%, acb= 23,85%
2. XNOR-NOR-XNOR			
XNOR-NOR-XNOR, ex. 1000, m=3, n=3	sat=100%, usat=0%, acb= 46,73%	epochs=1: sat=44,44%, usat=55,56%, acb= 56,59%	sat=100%, usat=0%, acb= 46,73%
XNOR-NOR-XNOR, ex. 5000, m=3, n=3	sat=100%, usat=0%, acb= 45,61%	epochs=1: sat=77,78%, usat=22,22%, acb= 55,18%	sat=100%, usat=0%, acb= 45,61%
XNOR-NOR-XNOR, ex. 5000, m=5, n=3	sat=100%, usat=0%, acb= 48,77%	epochs=1: sat=46,67%, usat=53,33%, acb= 49,63%	sat=100%, usat=0%, acb= 48,77%
XNOR-NOR-XNOR, ex. 10 000, m=5, n=3	sat=100%, usat=0%, acb= 48,73%	epochs=1: sat=53,33%, usat=46,67%, acb= 51,76%	sat=100%, usat=0%, acb= 48,73%
3. NAND-XNOR-NOR			
NAND-XNOR-NOR, ex. 1000, m=3, n=3	sat=100%, usat=0%, acb= 41,78%	epochs=1: sat=55,56%, usat=44,44%, acb= 49,08%	sat=100%, usat=0%, acb= 41,78%
NAND-XNOR-NOR, ex. 100, m=3, n=3	sat=100%, usat=0%, acb= 33,19%	epochs=1: sat=0%, usat=100%, acb= 66,81%	sat=100%, usat=0%, acb= 33,19%
NAND-XNOR-NOR, ex. 5000, m=5, n=3	sat=100%, usat=0%, acb= 41,92%	epochs=1: sat=22,22%, usat=77,78%, acb= 57,28 %	sat=100%, usat=0%, acb= 41,92%
NAND-XNOR-NOR, ex. 5000, m=5, n=3	sat=100%, usat=0%, acb= 47,34%	epochs=1: sat=33,33%, usat=66,67%, acb= 48,16%	sat=100%, usat=0%, acb= 47,34%
NAND-XNOR-NOR, ex. 10 000, m=5, n=3	sat=100%, usat=0%, acb= 47,29%	epochs=1: sat=40%, usat=60%, acb= 50,28%	sat=100%, usat=0%, acb= 47,29%
Multi layer (n=5)			
XNOR-NOR-XNOR-NOR-XNOR, ex. 50, m=5, n=5	sat=100%, usat=0%, acb= 43,13%	epochs=1: sat=28%, usat=72%, acb= 42,58%	sat=100%, usat=0%, acb= 43,13%
XNOR-NOR-XNOR-NOR-XNOR, ex. 100, m=5, n=5	sat=100%, usat=0%, acb= 50%	epochs=1: sat=24%, usat=76%, acb= 57,69%	sat=100%, usat=0%, acb= 50%
XNOR-NOR-XNOR-NOR-XNOR, ex. 100, m=5, n=5	sat=100%, usat=0%, acb= 50%	epochs=1: sat=36%, usat=64%, acb= 53,33%	sat=100%, usat=0%, acb= 50%
XNOR-NOR-XNOR-NOR-XNOR, ex. 1000, m=5, n=5	sat=100%, usat=0%, acb= 49,49%	epochs=1: sat=56%, usat=44%, acb= 47,76%	sat=100%, usat=0%, acb= 49,49%
XNOR-NOR-XNOR-NOR-XNOR, ex. 1000, m=5, n=5	sat=100%, usat=0%, acb= 49,48%	epochs=1: sat=32%, usat=68%, acb= 51,91%	sat=100%, usat=0%, acb= 49,48%
NAND-NOR-NAND-NOR-NAND, ex. 100, m=5, n=5	sat=100%, usat=0%, acb= 50%	epochs=1: sat=68%, usat=32%, acb= 40%	sat=100%, usat=0%, acb= 50%
NAND-NOR-NAND-NOR-NAND, ex. 100, m=5, n=5	sat=100%, usat=0%, acb= 46,5%	epochs=1: sat=32%, usat=68%, acb= 55,21%	sat=100%, usat=0%, acb= 50%
NAND-NAND-NAND-NAND-NAND, ex. 1000, m=5, n=5	sat=100%, usat=0%, acb= 24,15%	epochs=1: sat=32%, usat=68%, acb= 71,77%	sat=100%, usat=0%, acb= 24,15%

Experiments – Comments (1)

Permutation MaxSat - target vrs prediction



```
TESTING OF MY NET ([data, label]) ==>
+-----+
METHOD TESTED: Permutation MaxSat
BALANCED ACCURACY obtained: 100.00 %
+-----+
Number of examples in the TEST SET: 30
Number of tested examples: 30
OKAY, Net TESTED :-> <<
```

Thanks all these experiments I was able to conclude that:

Permutation MaxSat and Clauses MaxSat algorithms follow a more structured logic than Naïve Search.

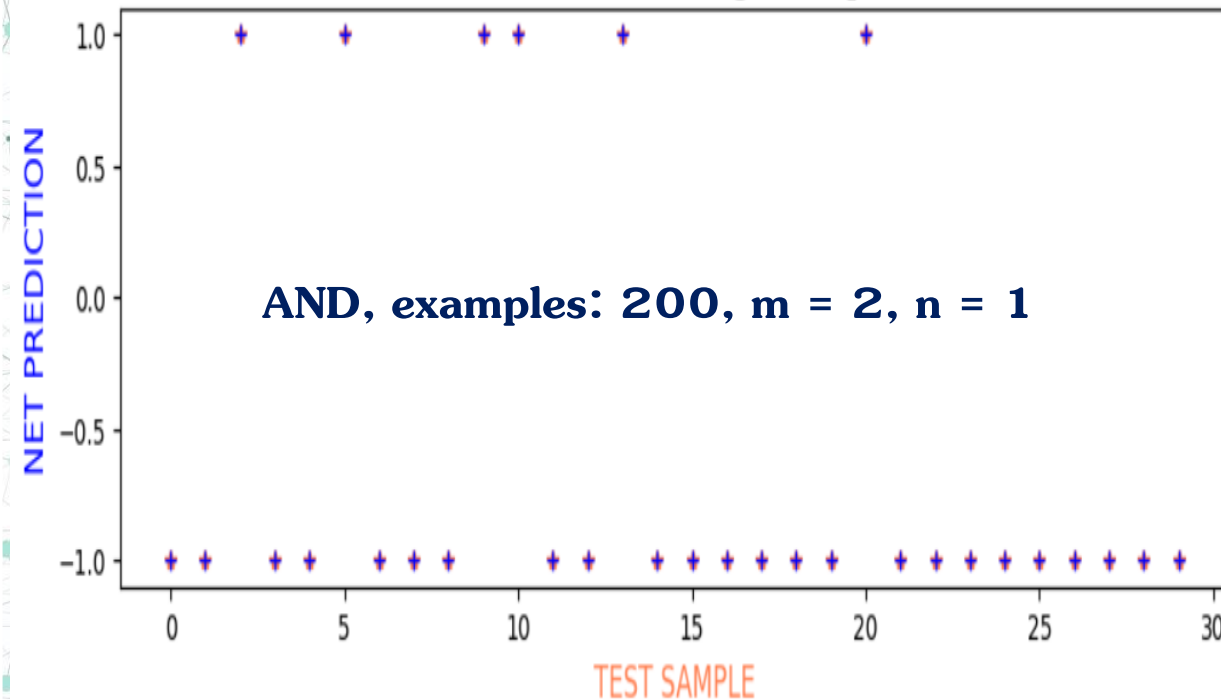
Indeed



1. In general, the **more complex the net architecture**, the **more difficult it is to obtain good performance** (i.e more examples, greater use of resources);
2. The Naïve Search algorithm is **less robust** than the others two in term of accuracy (i.e more influenced by the runtime);
3. The implemented binary functions **influence the difficulty** of solving the Maxsat problem;

Experiments – Comments (1)

Permutation MaxSat - target vrs prediction



```
TESTING OF MY NET ([data, label]) ==>
+-----+
METHOD TESTED: Permutation MaxSat
BALANCED ACCURACY obtained: 100.00 %
+-----+
Number of examples in the TEST SET: 30
Number of tested examples: 30
OKAY, Net TESTED :-> <<
```

Thanks all these experiments I was able to conclude that:

Permutation MaxSat and Clauses MaxSat algorithms follow a more structured logic than Naïve Search.

Indeed



1. In general, the **more complex the net architecture**, the **more difficult it is to obtain good performance** (i.e more examples, greater use of resources);
2. The Naïve Search algorithm is **less robust** than the others two in term of accuracy (i.e more influenced by the runtime);
3. The im of solv

AND, OR, NOT	Low difficulty
NAND, NOR	Medium difficulty
XOR, XNOR	Hight difficulty

the difficulty

Experiments – Comments (2)

Thanks all these experiments I was able to conclude that:

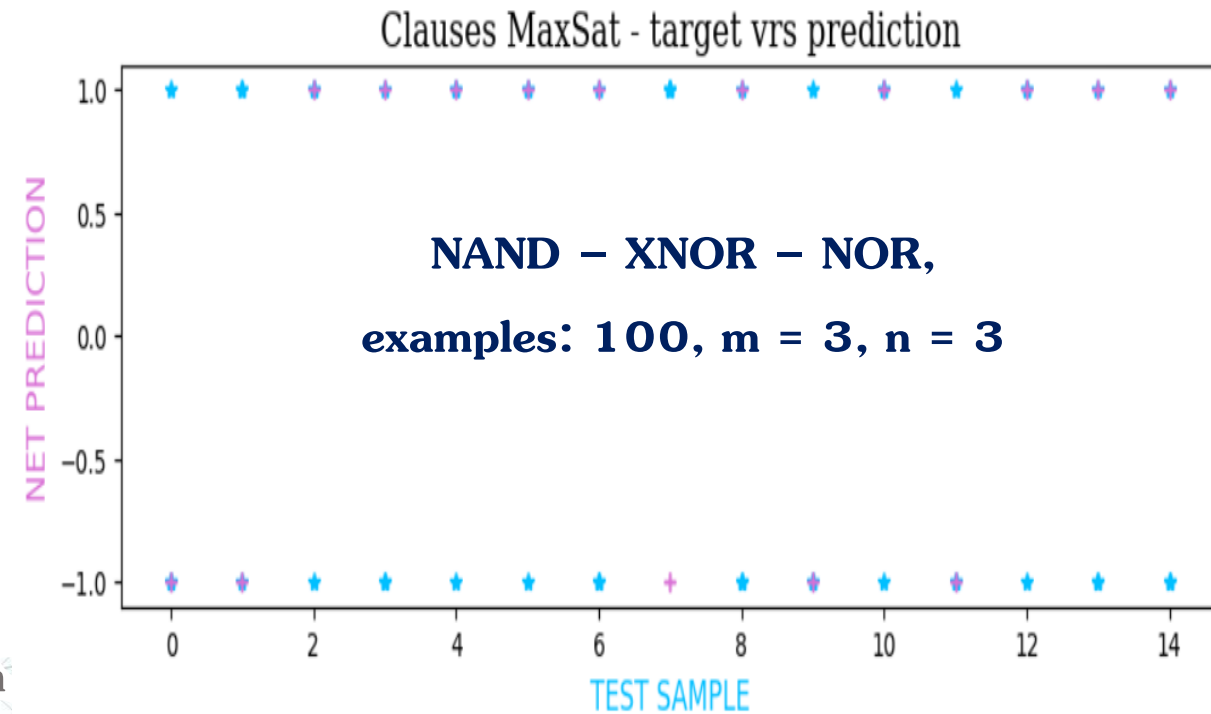
Permutation MaxSat and Clauses MaxSat algorithms follow a more structured logic than Naïve Search.

Indeed



4. There are cases in which if the net is too simple the Permutation and Clauses MaxSat algorithms can give a **poor performance** (i.e they are too elaborate for the net itself). In contrast, the Naïve Search can reach **better results**.

The Naïve Search obtains a result better: 66.60%



```
TESTING OF MY NET ([data, label]) =====>
+-----+
METHOD TESTED: Clauses MaxSat
BALANCED ACCURACY obtained: 48.52 %
+-----+
Number of examples in the TEST SET: 15
Number of tested examples: 15
OKAY, Net TESTED :-> <<
```

DEMO – Test in real time!

- ❖ **Easy net architecture:**
 - m input neurons, **1** binary function and $n=1$ output neuron;
- ❖ **Medium net architecture:**
 - m input neurons, **3** binary functions and $n=3$ output neurons;
- ❖ **Complex net architecture:**
 - m input neurons, **5** binary functions and $n=5$ output neurons.

- ✓ **Changing the number of examples, the number of m and with different combinations of binary functions;**
- ✓ **Both for the no hidden layers and the hidden layers versions.**

Thanks for your attention!!!

Thanks for your attention!!!

QUESTION TIME

